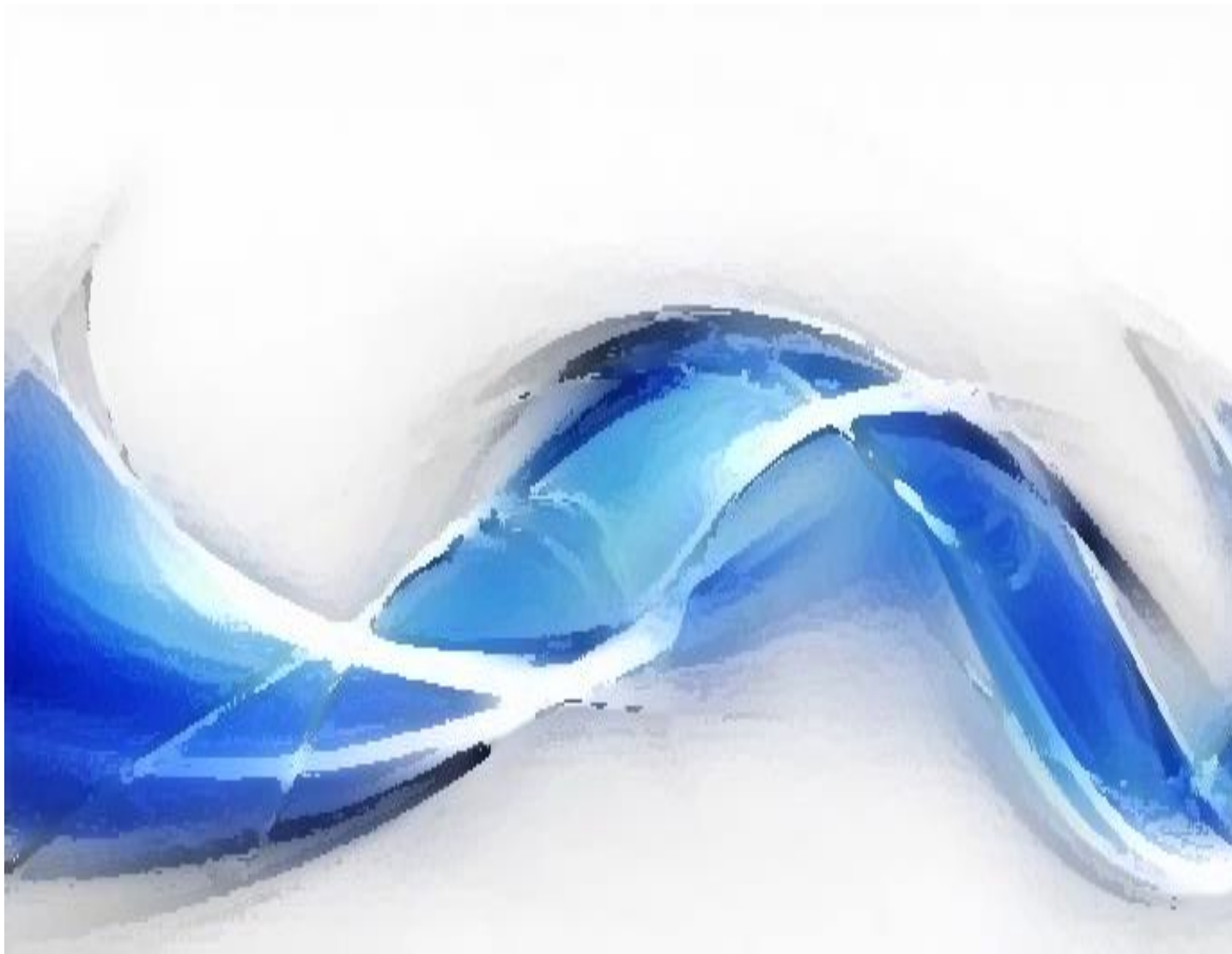




Luciana Ribeiro Veloso

luciana.veloso@dee.ufcg.edu.br

Processamento Digital de Sinais - Aula 08

Transformada Discreta da Fourier – Representação Matricial

TRANSFORMADA DISCRETA DE FOURIER - DFT

- Se $x(n)$ é um sinal de tempo discreto, causal e de comprimento L

Transformada discreta de Fourier

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j\frac{2\pi k}{N}n}$$



Eq. de Análise

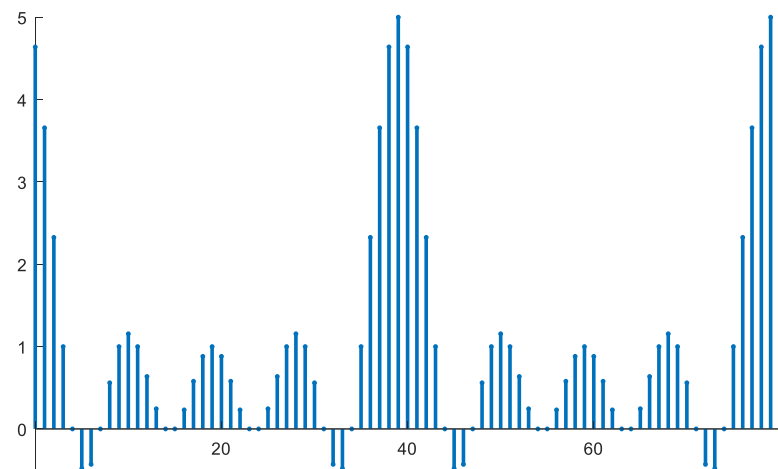
N é o número de amostras da DFT e $N \geq L$

$$x(n) = \begin{cases} \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j\frac{2\pi k}{N}n}, n = 0, 1, \dots, N-1 \\ 0, \text{ caso contrário} \end{cases}$$

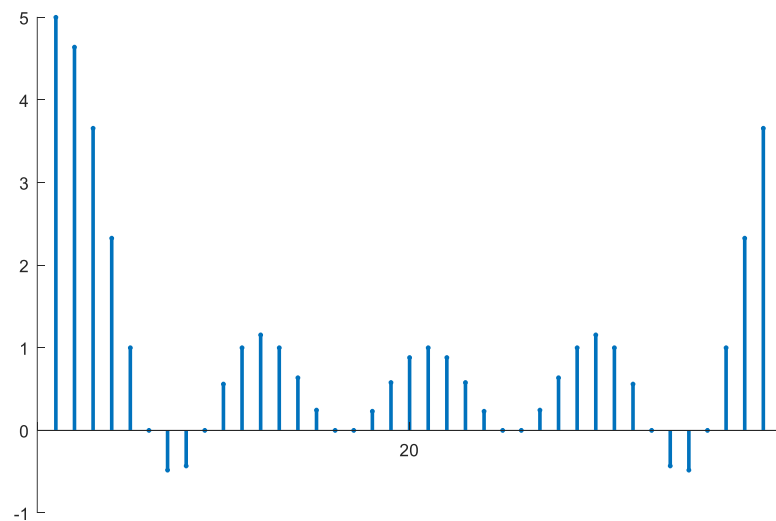


Eq. de síntese

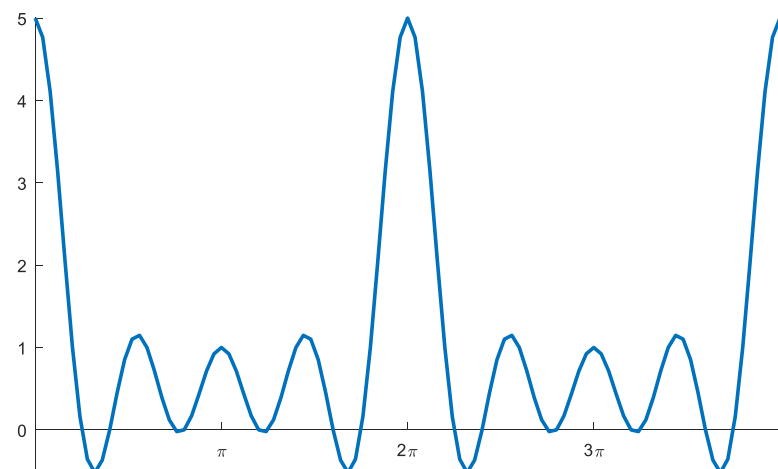
SÉRIE, TRANSFORMADA E DFT



Série



DFT de 40 pontos



Transformada

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j\frac{2\pi k}{N}n}$$

- Para simplificar a notação, define-se:

$$W_N = e^{-j\frac{2\pi}{N}}$$

- As transformadas podem ser escritas como:

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn}, k = 0, 1, \dots, N-1$$

$$x(n) = \sum_{K=0}^{N-1} X(k) W_N^{-kn}, n = 0, 1, \dots, N-1$$

EXEMPLO

- Calcule a DFT com $N = 4$ pontos da sequência $x(n) = \{0, 1, 2, 3\}$

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{kn}, k = 0, 1, \dots, N-1$$

$$W_N = e^{-j\frac{2\pi}{N}}$$

EXEMPLO (DFT N=4)

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{kn}, k = 0, 1, \dots, N-1$$

$$X(0) = x(0)W_4^{0.0} + x(1)W_4^{0.1} + x(2)W_4^{0.2} + x(3)W_4^{0.3}$$

$$X(1) = x(0)W_4^{1.0} + x(1)W_4^{1.1} + x(2)W_4^{1.2} + x(3)W_4^{1.3}$$

$$X(2) = x(0)W_4^{2.0} + x(1)W_4^{2.1} + x(2)W_4^{2.2} + x(3)W_4^{2.3}$$

$$X(3) = x(0)W_4^{3.0} + x(1)W_4^{3.1} + x(2)W_4^{3.2} + x(3)W_4^{3.3}$$

EXEMPLO (DFT N=4)

$$X(k) = \sum_{n=0}^3 x(n) W_N^{kn}, k = 0, 1, \dots, 3$$

$$X(0) = x(0)W_4^{0.0} + x(1)W_4^{0.1} + x(2)W_4^{0.2} + x(3)W_4^{0.3}$$

$$X(1) = x(0)W_4^{1.0} + x(1)W_4^{1.1} + x(2)W_4^{1.2} + x(3)W_4^{1.3}$$

$$X(2) = x(0)W_4^{2.0} + x(1)W_4^{2.1} + x(2)W_4^{2.2} + x(3)W_4^{2.3}$$

$$X(3) = x(0)W_4^{3.0} + x(1)W_4^{3.1} + x(2)W_4^{3.2} + x(3)W_4^{3.3}$$

$$\Rightarrow \bar{X} = \bar{W}_N \cdot \bar{x}$$

$$\bar{W}_N = \begin{bmatrix} W_4^{0.0} & W_4^{0.1} & W_4^{0.2} & W_4^{0.3} \\ W_4^{1.0} & W_4^{1.1} & W_4^{1.2} & W_4^{1.3} \\ W_4^{2.0} & W_4^{2.1} & W_4^{2.2} & W_4^{2.3} \\ W_4^{3.0} & W_4^{3.1} & W_4^{3.2} & W_4^{3.3} \end{bmatrix} \quad \bar{x} = \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix} \quad \bar{X} = \begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix}$$

EXEMPLO (DFT N=4)

$$\bar{W}_4 = \begin{bmatrix} W_4^{0.0} & W_4^{0.1} & W_4^{0.2} & W_4^{0.3} \\ W_4^{1.0} & W_4^{1.1} & W_4^{1.2} & W_4^{1.3} \\ W_4^{2.0} & W_4^{2.1} & W_4^{2.2} & W_4^{2.3} \\ W_4^{3.0} & W_4^{3.1} & W_4^{3.2} & W_4^{3.3} \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^4 & W_4^6 \\ 1 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix}$$

$$W_N = e^{-j2\pi/N}$$

$$W_4 = e^{-j2\pi/4} = e^{-j\pi/2} = \cos\left(\frac{\pi}{2}\right) - j\sin\left(\frac{\pi}{2}\right) = -j$$

$$\bar{W}_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

EXEMPLO

- Calcule a DFT com 4 pontos da sequência $x(n) = \{0, 1, 2, 3\}$

$$\overline{W}_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

$$\bar{x} = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}$$

$$\bar{X} = \overline{W}_N \bar{x} =$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

$$\cdot \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 6 \\ -2 + 2j \\ -2 \\ -2 - 2j \end{bmatrix}$$

DFT E IDFT

- Em notação matricial

$$\bar{X} = \bar{W}_N \bar{x}$$

$$\bar{x} = \frac{1}{N} \bar{W}_N^* \bar{X}$$

$$\bar{x} = \begin{bmatrix} x[0] \\ x[1] \\ \vdots \\ x[N-1] \end{bmatrix}$$

$$\bar{X} = \begin{bmatrix} X[0] \\ X[1] \\ \vdots \\ X[N-1] \end{bmatrix}$$

$$\bar{W}_N = \begin{bmatrix} W_N^0 & W_N^0 & \dots & W_N^0 \\ W_N^0 & W_N^1 & \dots & W_N^{N-1} \\ \vdots & \vdots & \dots & \vdots \\ \vdots & \vdots & \dots & \vdots \\ W_N^0 & W_N^{N-1} & \dots & W_N^{(N-1)(N-1)} \end{bmatrix}$$

A vantagem desse método é que ele é útil em sistemas baseados em representação matricial, Ex. MATLAB e hardware dedicado.

CUSTO COMPUTACIONAL

$$X(0) = x(0)W_4^{0.0} + x(1)W_4^{0.1} + x(2)W_4^{0.2} + x(3)W_4^{0.3}$$

$$X(1) = x(0)W_4^{1.0} + x(1)W_4^{1.1} + x(2)W_4^{1.2} + x(3)W_4^{1.3}$$

$$X(2) = x(0)W_4^{2.0} + x(1)W_4^{2.1} + x(2)W_4^{2.2} + x(3)W_4^{2.3}$$

$$X(3) = x(0)W_4^{3.0} + x(1)W_4^{3.1} + x(2)W_4^{3.2} + x(3)W_4^{3.3}$$

- Para realizar o calculo cada valor $X(k)$ são necessários
 - 4 multiplicações complexas
 - 3 adições complexas.
- Para obter todos N valores $X[k]$ são necessários
 - **4^2 multiplicações complexas**, e
 - **$(3)4$ adições complexas.**

CUSTO COMPUTACIONAL DE DFT N PONTOS

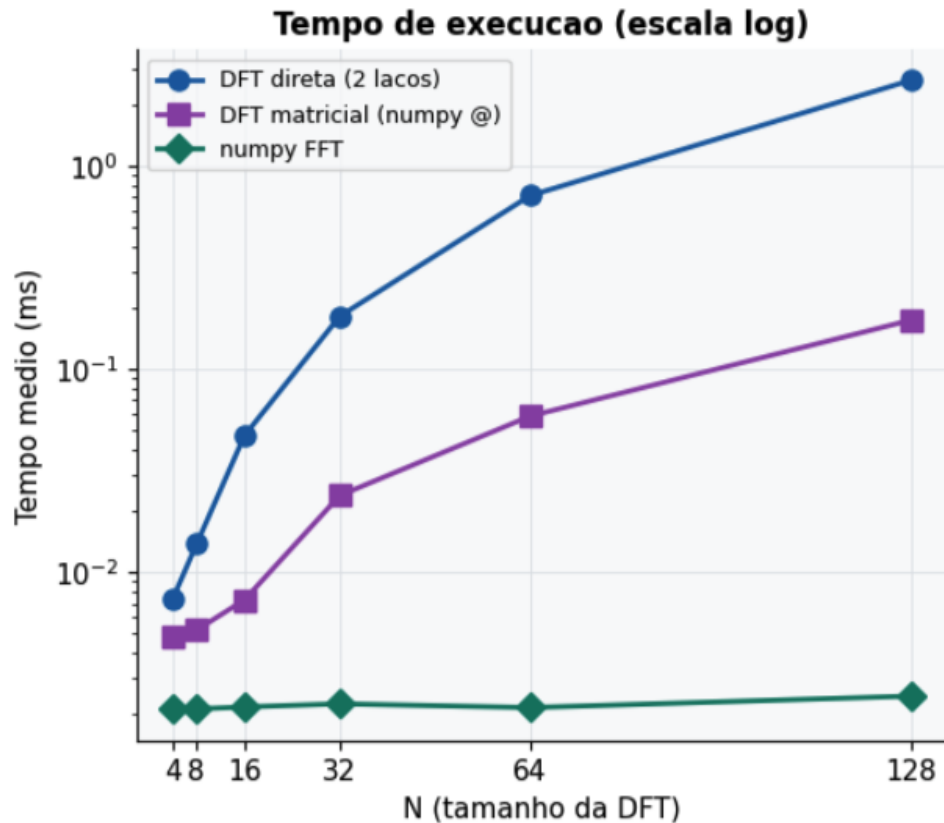
- Para realizar o calculo cada valor $X(k)$ são necessários
 - N multiplicações complexas
 - $N - 1$ adições complexas.
- Para obter todos N valores $X[k]$ são necessários
 - **n^2 multiplicações complexas, e**
 - **$(N - 1)N$ adições complexas.**

Por isso dizemos que a DFT tem complexidade $O(N^2)$

Para áudio com $F_s = 44.100$ Hz e janela de 1s: $N = 44.100$, custo $\approx 2 \times 10^9$ operações, inviável em tempo real.

Motivação direta para o algoritmo FFT (que reduz a $O(N \log_2 N)$).

COMPARATIVO DO TEMPO MÉDIO DE EXECUÇÃO DA DFT



Resultados idênticos

As três implementações produzem exatamente o mesmo resultado — o erro entre elas é da ordem de 10^{-14} , apenas ruído de arredondamento de ponto flutuante. São três formas de calcular a mesma operação matemática.

Tempos muito diferentes

Embora direta e matricial tenham o mesmo custo teórico $O(N^2)$, a versão matricial é significativamente mais rápida porque o numpy executa o produto matricial em código C otimizado, sem o overhead dos laços Python. A FFT é ainda mais rápida por usar $O(N \log_2 N)$ operações.

Laços em Python são lentos

A DFT direta com dois laços em Python puro é muito lenta porque cada iteração tem overhead de interpretação. Para $N=64$, ela é dezenas de vezes mais lenta que a versão matricial, que por sua vez ainda é muito mais lenta que a FFT.